

BÀI TẬP VỀ NHÀ (NestJS) — Số 01: Recipe API + MongoDB

Phạm vi ôn tập: Controllers & Routing · Persistence (Mongoose) · Tích hợp MongoDB thật — đúng những gì bạn vừa xây dựng thật trong `hoc-nestjs` (Recipe CRUD + Atlas).

Nguyên tắc quan trọng: KHÔNG động vào project `hoc-nestjs`. Nó đang là backend **sống** của Recipe App (Angular gọi thẳng vào nó, dữ liệu thật nằm trên MongoDB Atlas thật). Toàn bộ bài tập làm trên project riêng `bai-tap-tra-sua-api`.

Bộ này viết cho phiên bản nào

Xác minh trực tiếp trong `hoc-nestjs/package.json` lúc soạn bài: **NestJS 11.0.x** · **@nestjs/mongoose 11.0.4** · **mongoose 9.9.2** · **@nestjs/config 4.0.4** · **Node 24**. Nếu máy bạn khác bản, tra lại API trong `node_modules` trước khi làm.

Vì sao chọn domain "Quầy Trà Sữa"

Bạn đã dùng domain này ở `BAI-TAP-VE-NHA-17-21.md` bên Angular (DrinkModel, Topping, project `bai-tap-tra-sua`). Bộ này xây **backend thật** cho đúng domain đó — cùng tên field, cùng dữ liệu mẫu. Xong Phần D bạn có thể **nối hai project luyện tập lại với nhau**, đóng vòng full-stack y hệt cách Recipe App đã làm, mà không đụng gì đến project chính.

Nguyên tắc soạn Phần A

Câu hỏi lý thuyết **không hỏi tên method của Mongoose hay tên decorator của Nest**. Loại kiến thức đó chỉ đúng với một thư viện, đổi sang TypeORM, Prisma, hay Express thuần là bỏ đi.

Câu hỏi tập trung vào **nguyên tắc mang đi được**: kiến trúc phân lớp, REST, seam giữa các tầng, vì sao cấu hình phải fail sớm. Mongoose/Nest chỉ là **ví dụ minh họa**.

Tên API cụ thể nằm ở **sổ tay tra cứu cuối file** — để tra khi làm bài, không phải học thuộc.

Chuẩn bị

```
cd <thư mục chứa project của bạn>
nest new bai-tap-tra-sua-api --package-manager npm --skip-git
cd bai-tap-tra-sua-api
```

Cài các gói cần cho bài này (đúng bộ bạn vừa dùng thật ở `hoc-nestjs`):

```
npm install @nestjs/mongoose mongoose @nestjs/config
```

⚠️ **Commit Git trước khi bắt đầu.** `nest generate` sinh nhiều file cùng lúc. Có gì sai thì `git checkout .` là về nguyên trạng.

```
git init && git add -A && git commit -m "khởi tạo bai-tap-tra-sua-api"
```

MongoDB: dùng chung tài khoản Atlas bạn vừa tạo, nhưng **tạo database mới** tên `bai-tap-tra-sua` (không lưu chung database với `recipe-app` của project chính — mai làm sai gì ở bài tập không ảnh hưởng dữ liệu thật). Connection string giữ nguyên cluster, chỉ đổi tên database trong URI: `.../bai-tap-tra-sua?retryWrites=true&w=majority`.

📦 Domain khởi đầu

Đây là hình dạng dữ liệu — bạn tự viết code, không copy-paste. Khớp đúng `DrinkModel` bạn đã dùng bên Angular để cuối bài nối được hai project luyện tập lại với nhau.

```
// Hình dạng dữ liệu cần đạt được (viết ra để tham khảo, KHÔNG phải code Nest thật)
interface Topping {
  name: string;
  quantity: number;
  unit: string;
}

interface DrinkModel {
  id: string; // ObjectId của Mongo, dạng chuỗi
  name: string;
  description: string;
  giaCoBan: number;
  imgUrl: string;
  isPopular: boolean;
  toppings: Topping[];
  authorEmail: string;
}
```

Dữ liệu mẫu để seed (3 món, giữ nguyên từ bên Angular để hai phía khớp nhau):

```
export const SEED_DRINKS = [
  {
    name: 'Trà sữa trân châu đường đen',
    description: 'Trà sữa đậm vị, trân châu dai mềm ngâm đường đen.',
    giaCoBan: 45000,
    imgUrl: 'https://images.unsplash.com/photo-1558857563-b371033873b8?w=600',
    isPopular: true,
    authorEmail: 'quay@trasua.com',
    toppings: [
```

```

    { name: 'Trân châu đường đen', quantity: 50, unit: 'g' },
    { name: 'Sữa tươi', quantity: 100, unit: 'ml' },
  ],
},
{
  name: 'Trà sữa Matcha',
  description: 'Matcha Nhật nguyên chất, hơi đắng nhẹ, thơm dịu.',
  giaCoBan: 52000,
  imgUrl: 'https://images.unsplash.com/photo-1515823064-d6e0c04616a7?w=600',
  isPopular: true,
  authorEmail: 'quay@trasua.com',
  toppings: [
    { name: 'Bột matcha', quantity: 8, unit: 'g' },
    { name: 'Sữa tươi', quantity: 120, unit: 'ml' },
  ],
},
{
  name: 'Hồng trà sữa',
  description: 'Hồng trà truyền thống pha sữa, vị cân bằng dễ uống.',
  giaCoBan: 35000,
  imgUrl: 'https://images.unsplash.com/photo-1571934811356-5cc061b6821f?w=600',
  isPopular: false,
  authorEmail: 'quay@trasua.com',
  toppings: [
    { name: 'Hồng trà', quantity: 10, unit: 'g' },
    { name: 'Thạch dừa', quantity: 40, unit: 'g' },
  ],
},
];

```

BÀI GIẢNG BỔ SUNG — Vì sao lại tách lớp thế này?

Phần này giải thích **lý do**, không phải cú pháp. Đọc trước Phần A.

Bốn lớp trong một request

Một request **GET /drinks/:id** đi qua bốn lớp, mỗi lớp chỉ biết một việc:

Controller	→	Service	→	(Repository)	→	Model của Mongoose	→	MongoDB Atlas
"nhận HTTP"		"nghịệp vụ"		"nói chuyện DB"		"dịch sang Mongo"		"lưu thật"

Bạn đã tự tay xây ba lớp đầu (Controller/Service/Model) khi làm Recipe API. Lớp **Repository** (trong ngoặc) là phần **chưa có** — đó là nội dung mới của bài này, ở Phần D.

Vì sao phải tách — khái niệm "seam" (đường nối)

Seam là một điểm trong code mà bạn **đổi được một bên** mà **bên còn lại không hay biết**.

`RecipeService` hiện tại:

```
constructor(@InjectModel(Recipe.name) private readonly recipeModel:
Model<RecipeDocument>) {}
```

Ở đây **không có seam**. `RecipeService` biết rõ mình đang nói chuyện với Mongoose — gọi `.find()`, `.findById()`, `.exec()` — toàn bộ là API của Mongoose. Mai muốn đổi sang PostgreSQL, bạn phải **sửa từng dòng trong service**.

Nếu thay bằng một interface đứng giữa:

```
interface DrinkRepository {
  findAll(keyword?: string): Promise<Drink[]>;
  findOne(id: string): Promise<Drink | null>;
  // ...
}
```

`DrinkService` chỉ gọi `this.repo.findAll()` — **không biết** bên dưới là Mongoose, PostgreSQL, hay một mảng RAM. Đó chính là **seam**: đổi implementation, `DrinkService` không sửa một chữ.

🎓 Đây là nguyên tắc **Dependency Inversion** (chữ D trong SOLID) — tầng nghiệp vụ phụ thuộc vào một **giao diện trừu tượng**, không phụ thuộc vào một thư viện cụ thể. Áp dụng được ở Java (Spring), C# (.NET), Python (Django) — không riêng gì NestJS.

Injection token — vì sao cần, khi interface "biến mất" lúc chạy

TypeScript compile xong thì **interface không còn tồn tại** trong file JavaScript (chỉ có tác dụng lúc biên dịch, để bắt lỗi kiểu). NestJS DI hoạt động lúc **runtime**, nên nó không thể "tìm theo tên interface" được nữa.

Giải pháp: tạo một **token** thay mặt cho interface đó — có thể là chuỗi hoặc `Symbol`:

```
export const DRINK_REPOSITORY = Symbol('DRINK_REPOSITORY');
```

Đăng ký trong module: "ai xin token này, đưa cho họ `MongooseDrinkRepository`":

```
{ provide: DRINK_REPOSITORY, useClass: MongooseDrinkRepository }
```

Trong service, xin bằng đúng token đó, nhưng khai kiểu bằng **interface**:

```
constructor(@Inject(DRINK_REPOSITORY) private readonly repo:
DrinkRepository) {}
```

`@Inject(...)` nói **runtime** đi tìm token nào. Kiểu `DrinkRepository` nói **compile-time** service này được phép gọi những method nào. Hai thứ khác nhau, cùng tồn tại song song.

Mongoose Schema — bảo vệ ở tầng nào?

MongoDB tự thân **không ép schema** — bạn có thể lưu 2 document cùng collection với 2 hình dạng field khác hẳn nhau, Mongo không phàn nàn gì. Vậy `@Schema()`/`@Prop()` của Mongoose đang bảo vệ ở đâu?

Trả lời: ở tầng ứng dụng (application layer), không phải tầng database. Mongoose kiểm tra **required**, kiểu dữ liệu... **trước khi** gửi lệnh xuống Mongo. Bạn được lợi ích của schema (bắt lỗi sớm, gợi ý kiểu trong TypeScript) mà **không mất** tính linh hoạt của MongoDB — vẫn có thể thêm field mới vào schema bất cứ lúc nào mà không cần "migration" như SQL.

toJSON transform — vì sao không trả thẳng `_id`

Mongo tự sinh khóa chính tên `_id`, kiểu `ObjectId` — không phải `string`, không phải `id`. Trả thẳng ra ngoài thì:

1. Frontend phải biết về khái niệm `ObjectId` của Mongo — **rò rỉ chi tiết implementation** của database ra tận giao diện người dùng.
2. Field tên `_id` (có gạch dưới) không tự nhiên với REST API — client mong đợi `id`.

`transform` trong `toJSON` của schema là nơi "dịch" `_id` (`ObjectId`) → `id` (`string`) đúng một lần, tại biên giới giữa app và bên ngoài — y hệt lý do **DTO** tồn tại: **không để chi tiết bên trong rò ra ngoài**.

CastError — vì sao id sai định dạng làm app "nổ"

`findById("khong-phai-object-id")` không trả `null` như "tìm không thấy" — nó **ném lỗi ngay**, vì Mongoose thử "ép kiểu" (cast) chuỗi đó thành `ObjectId` trước khi gửi query, và phép ép kiểu đó thất bại. Nếu không bắt lỗi này, Nest trả về **500 Internal Server Error** — sai hoàn toàn, vì đây không phải lỗi server, mà là **client gửi id sai định dạng** (đáng lẽ phải là 404).

Cách phòng: kiểm tra định dạng **trước khi** query, dùng `Types.ObjectId.isValid(id)`.



PHẦN A — Lý thuyết

Câu hỏi tập trung vào **nguyên tắc mang đi được** — vẫn đúng khi bạn đổi sang Express thuần, Spring Boot, hay Django. Không có câu nào bắt học thuộc tên method của Mongoose.

Về kiến trúc NestJS nói chung

1. `@Injectable()` cho phép Nest "tự tạo và tự truyền" một instance mà bạn không hề gọi `new`. Cơ chế nào đứng sau chuyện đó? (Gợi ý: liên hệ khái niệm **container quản lý vòng đời**.)

2. `@Module()` có cả `providers` lẫn `exports`. Một provider nằm trong `providers` nhưng **không** nằm trong `exports` thì module khác import được xây ra chuyện gì khi cố `inject` nó? Vì sao thiết kế "mặc định đóng, phải khai báo mới mở" lại an toàn hơn "mặc định mở"?
3. Vì sao logic nghiệp vụ (business logic) không nên nằm trong Controller? Nêu **hai** hậu quả cụ thể nếu bạn viết thẳng code truy vấn database trong hàm xử lý route.

Về REST API design (mang đi được)

4. Tạo tài nguyên thành công thường trả **201**, không phải **200**. Sự khác biệt đó nói lên điều gì với client gọi API? Khi nào một endpoint POST **vẫn nên** trả 200 thay vì 201?
5. Một request `GET /drinks/:id` với id **không tồn tại** và một request với id **sai định dạng** (ví dụ `"abc"`) — vì sao cả hai nên trả cùng **404**, thay vì cái sau trả 400? (Gợi ý: nghĩ về việc client bên ngoài có cần biết bạn dùng ObjectId hay số nguyên hay không.)
6. Route param (`:id`) và query param (`?keyword=`) đều mang dữ liệu từ URL. Nêu quy tắc chọn cái nào — liên hệ với sự khác biệt giữa "định danh một tài nguyên cụ thể" và "lọc/tùy chọn trên một tập hợp".

Về MongoDB / Mongoose (nguyên tắc, không phải cú pháp)

7. Cơ sở dữ liệu quan hệ tổ chức dữ liệu thành **bảng - hàng - cột**, có khóa ngoại nối bảng này với bảng khác. MongoDB tổ chức thành **collection - document**. Nêu **một** tình huống dữ liệu (ví dụ: `Recipe` có mảng `ingredients`) mà mô hình document xử lý gọn hơn mô hình quan hệ, và giải thích vì sao.
8. `@Schema()`/`@Prop()` của Mongoose trông giống hệt việc "ép schema" — nhưng MongoDB tự thân không ép schema ở tầng lưu trữ. Vậy Mongoose Schema đang bảo vệ ở **tầng nào**? Nêu một lợi ích cụ thể bạn được nhờ điều đó, và một thứ bạn **không** mất đi so với dùng MongoDB thuần.
9. Vì sao ứng dụng không nên trả thẳng field `_id` (kiểu `ObjectId`) ra API cho client, mà cần "transform" thành `id` dạng chuỗi? Liên hệ với lý do DTO tồn tại (Bài nào bạn học về DTO thì liên hệ lại — kể cả khi bài đó bạn chưa làm ở project này).
10. `findById("không-hop-le")` ném `CastError` thay vì trả `null`. Nêu hậu quả nếu không bắt lỗi này, và cách phòng tránh mà không cần thư viện validate ngoài.

Về cấu hình (config) & khởi động ứng dụng

11. Connection string MongoDB (chứa mật khẩu) nằm trong `.env`, không hardcode trong code — kể cả khi project chỉ chạy trên máy cá nhân, không public. Nêu **hai** lý do, một lý do về bảo mật và một lý do **không liên quan gì đến bảo mật**.
12. `onModuleInit()` chỉ seed dữ liệu mẫu khi `countDocuments() === 0`. Mô tả cụ thể chuyện gì xảy ra nếu bỏ điều kiện đó và seed vô điều kiện mỗi lần app khởi động lại (ví dụ: mỗi lần `npm run start:dev` tự restart do đổi code).

Về Repository Pattern (*nội dung mới của bài này — Phần D*)

13. Định nghĩa "seam" (đường nối) trong kiến trúc phần mềm bằng lời của riêng bạn. Vì sao `DrinkService` gọi thẳng `Model<DrinkDocument>` của Mongoose là **không có seam**, còn gọi qua interface `DrinkRepository` là **có seam**?
14. Sau khi compile TypeScript sang JavaScript, interface **biến mất hoàn toàn** — không còn dấu vết nào trong file chạy thật. Vậy NestJS dựa vào đâu để biết "ai cần cái gì" lúc chạy? Injection token giải quyết vấn đề đó thế nào?
15. Giả sử mai bạn đổi từ MongoDB sang PostgreSQL. Với cách viết "Service gọi thẳng Model của Mongoose", bạn phải sửa bao nhiêu file, và những file đó thuộc tầng nào? Với cách viết "qua interface + token", câu trả lời khác thế nào?

PHẦN B — Tìm lỗi trong code

Mỗi đoạn có **đúng một lỗi**, lấy cảm hứng từ những lỗi thật xảy ra lúc xây `hoc-nestjs`. Chỉ ra lỗi, giải thích **tại sao sai**, viết lại cho đúng.

Lỗi số 1

```
@Injectable()
export class DrinkService {
  constructor(private readonly drinkModel: Model<DrinkDocument>) {}

  findAll(): Promise<Drink[]> {
    return this.drinkModel.find().exec();
  }
}
```

Lỗi số 2

```
@Get('/:id')
findOne(@Param('id') id: string) {
  return this.drinkService.findOne(+id);
}
```

Lỗi số 3

```
async findOne(id: string): Promise<Drink> {
  const drink = await this.drinkModel.findById(id).exec();
  if (!drink) {
    throw new NotFoundException(`Drink with id ${id} not found`);
  }
  return drink;
}
```

Lỗi số 4

```
async onModuleInit(): Promise<void> {
  await this.drinkModel.insertMany(SEED_DRINKS);
}
```

Lỗi số 5

```
@Schema()
export class Drink {
  @Prop({ required: true })
  name!: string;

  @Prop({ type: [ToppingSchema], default: [] })
  toppings!: Topping[];
}
```

Lỗi số 6

```
@Schema()
export class Topping {
  @Prop({ required: true })
  name!: string;

  @Prop({ required: true })
  quantity!: number;
}

export const ToppingSchema = SchemaFactory.createClass(Topping);
```

Lỗi số 7

```
@Module({
  imports: [
    MongooseModule.forRootAsync({
```



```

        useFactory: () => ({ uri: process.env.MONGODB_URI }),
      },
    ],
  })
  export class AppModule {}

```

Lỗi số 8

```

@Delete('/:id')
@HttpCode(204)
remove(@Param('id') id: string) {
  this.drinkService.remove(id);
}

```

Lỗi số 9

```

@Module({
  providers: [{ provide: DrinkService, useClass: MongooseDrinkRepository }],
  controllers: [DrinkController],
})
export class DrinkModule {}

```

Lỗi số 10

```

@Injectable()
export class DrinkService {
  constructor(
    @Inject(DRINK_REPOSITORY) private readonly repo:
    MongooseDrinkRepository,
  ) {}
}

```



PHẦN C — Project: Quầy Trà Sữa API 🥤

Nhiệm vụ C-N1 — Khởi tạo & sinh module

```

nest g module drink
nest g controller drink --no-spec
nest g service drink --no-spec

```

Kiểm tra `app.module.ts` đã tự thêm `DrinkModule` vào `imports` chưa (Nest CLI làm hộ, nhưng luôn kiểm tra lại — đừng tin mù).

Nhiệm vụ C-N2 — Mongoose Schema

Viết `drink/entities/drink.entity.ts`:

- `Topping` là **subdocument schema** (`@Schema({ _id: false })`) — vì sao có `_id: false` thì tự trả lời ở Lỗi số 6 phía trên).
- `Drink` là schema chính, có `toJSON.transform` biến `_id` → `id` (string), xóa `__v`.
- Field: `name`, `description`, `giaCoBan` (number), `imgUrl`, `isPopular` (boolean, default `false`), `toppings` (mảng `ToppingSchema`, default `[]`), `authorEmail`.

Nhiệm vụ C-N3 — DTO

`create-drink.dto.ts` với các field ở trên (không bắt buộc dùng `class-validator` ở bài này — check tay đơn giản trong service như bạn đã làm ở Recipe là đủ). `update-drink.dto.ts` dùng `PartialType(CreateDrinkDto)`.

Nhiệm vụ C-N4 — Service

- `create`, `findAll(keyword?)` (tìm theo `name`, regex không phân biệt hoa/thường — y hệt Recipe), `findOne(id)`, `update(id, dto)`, `remove(id)`.
- Tất cả thao tác theo id đều kiểm tra `Types.ObjectId.isValid(id)` trước khi query.
- `onModuleInit()` seed `SEED_DRINKS` chỉ khi `countDocuments() === 0`.

Nhiệm vụ C-N5 — Controller

Map đủ 5 route: `POST /drinks`, `GET /drinks` (nhận `@Query('keyword')`), `GET /drinks/:id`, `PATCH /drinks/:id`, `DELETE /drinks/:id` (trả 204, nhớ `await`).

Nhiệm vụ C-N6 — Kết nối MongoDB Atlas thật

`app.module.ts`: `ConfigModule.forRoot({ isGlobal: true })` + `MongooseModule.forRootAsync` đọc `MONGODB_URI` qua `ConfigService`. Tạo `.env` + `.env.example`, thêm `.env` vào `.gitignore`.

🔗 **Tự kiểm tra:** khởi động app, log phải có dòng `MongooseCoreModule dependencies initialized` — đúng dấu hiệu bạn đã thấy thật ở `hoc-nestjs`.

Nhiệm vụ C-N7 — Test bằng curl (bắt buộc, không được bỏ qua)

Chạy đủ bộ sau, ghi lại kết quả:

```
# 1. Seed tự chạy — phải thấy 3 món
curl -s http://localhost:3001/drinks

# 2. Tạo món mới — phải trả 201 kèm "id" dạng ObjectId string
curl -s -X POST http://localhost:3001/drinks -H "Content-Type: application/json" \
```

```
-d '{"name":"Trà đào cam
sả","description":"...","giaCoBan":40000,"authorEmail":"a@b.com"}'

# 3. Tìm kiếm – chỉ trả món khớp
curl -s "http://localhost:3001/drinks?keyword=matcha"

# 4. Id sai định dạng – PHẢI trả 404, không phải 500
curl -s -w "\nstatus: %{http_code}\n" http://localhost:3001/drinks/khong-
phai-object-id

# 5. Xóa rồi gọi lại – phải trả 404
```

🎯 **Tự kiểm tra:** Bước 4 là bước quan trọng nhất — nó chứng minh bạn đã xử lý **CastError** đúng cách (Lỗi số 3 ở Phần B).

🚀 PHẦN D — Repository Pattern (BẮT BUỘC)

Đây là nội dung **mới**, chưa từng làm ở **hoc-nestjs**. Đọc lại mục "Vì sao phải tách — khái niệm seam" ở Bài giảng bổ sung trước khi làm.

D1 — Định nghĩa interface

Tạo **drink/drink.repository.interface.ts**:

```
import { Drink } from './entities/drink.entity';
import { CreateDrinkDto } from './dto/create-drink.dto';
import { UpdateDrinkDto } from './dto/update-drink.dto';

export interface DrinkRepository {
  create(dto: CreateDrinkDto): Promise<Drink>;
  findAll(keyword?: string): Promise<Drink[]>;
  findOne(id: string): Promise<Drink | null>;
  update(id: string, dto: UpdateDrinkDto): Promise<Drink | null>;
  remove(id: string): Promise<Drink | null>;
}

export const DRINK_REPOSITORY = Symbol('DRINK_REPOSITORY');
```

💡 Chú ý kiểu trả về: **Promise<Drink | null>**, không phải ném lỗi. **Ném NotFoundException là việc của Service** (tầng biết về HTTP), không phải của Repository (tầng chỉ biết về dữ liệu). Đây cũng là một seam — Repository không nên biết khái niệm "404" tồn tại.

D2 — Implementation bằng Mongoose

Tạo **drink/mongoose-drink.repository.ts**, class **MongooseDrinkRepository** implements **DrinkRepository**, **@Injectable()**, bên trong inject **@InjectModel(Drink.name)** như service cũ

của bạn — toàn bộ chi tiết Mongoose (`Types.ObjectId.isValid`, `.exec()`, regex tìm kiếm) chuyển vào **đây**, không còn ở Service nữa.

D3 — Bind token trong module

```
@Module({
  imports: [
    MongooseModule.forFeature([ { name: Drink.name, schema: DrinkSchema } ]),
  ],
  controllers: [DrinkController],
  providers: [
    DrinkService,
    { provide: DRINK_REPOSITORY, useClass: MongooseDrinkRepository },
  ],
})
export class DrinkModule {}
```

D4 — Service chỉ còn biết interface

```
@Injectable()
export class DrinkService {
  constructor(
    @Inject(DRINK_REPOSITORY) private readonly repo: DrinkRepository,
  ) {}

  async findOne(id: string): Promise<Drink> {
    const drink = await this.repo.findOne(id);
    if (!drink) {
      throw new NotFoundException(`Drink with id ${id} not found`);
    }
    return drink;
  }
  // ...
}
```

🔍 **Tự kiểm tra:** sau khi refactor, chạy lại **nguyên bộ curl ở C-N7** — kết quả phải giống hệt trước khi tách. Đây chính là ý nghĩa của "seam": hành vi bên ngoài **không đổi**, chỉ kiến trúc bên trong đổi.

Thử thách thêm (*bắt buộc*)

- **D5 — Chứng minh seam là thật.** Viết thêm `InMemoryDrinkRepository implements DrinkRepository` (mảng RAM, giống `DrinkService` hồi bạn học Bài 1–16 bên Angular). Đổi `useClass` trong `DrinkModule` sang class này — **không sửa gì ở Controller hay Service** — và chứng minh app vẫn chạy được (mất tính năng lưu vĩnh viễn, nhưng API vẫn hoạt động).
- **D6 — Nối hai project luyện tập.** Trò `bai-tap-tra-sua` (Angular, đã có `DrinkService` gọi mock) sang gọi API thật của `bai-tap-tra-sua-api` này qua `HttpClient/httpResource` — đúng hành

trình bạn đã đi với Recipe App, làm lại một lần nữa cho nhớ.

Bảng tự theo dõi tiến độ

Phần	Nội dung	Đã xong?
	Đọc bài giảng bổ sung (seam, token, schema)	☐
A	15 câu lý thuyết	☐
B	10 đoạn code tìm lỗi	☐
C-N1	Scaffold project + sinh module	☐
C-N2	Mongoose Schema (Topping + Drink)	☐
C-N3	DTO (create/update)	☐
C-N4	Service: CRUD + seed có điều kiện	☐
C-N5	Controller: 5 route	☐
C-N6	Kết nối MongoDB Atlas thật	☐
C-N7	Test đủ bộ curl, kể cả id sai định dạng	☐
D1-D4	Repository Pattern (bắt buộc)	☐
D5-D6	Thử thách thêm	☐

Sổ tay tra cứu nhanh

Mongoose Schema — cú pháp đã xác minh (mongoose 9.9.2, @nestjs/mongoose 11.0.4)

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document } from 'mongoose';

@Schema({ _id: false }) // subdocument: không cần _id riêng
export class Topping {
  @Prop({ required: true })
  name!: string;

  @Prop({ required: true })
  quantity!: number;

  @Prop({ required: true })
  unit!: string;
}
export const ToppingSchema = SchemaFactory.createForClass(Topping);
```

```

@Schema({
  toJSON: {
    virtuals: true,
    transform: (_doc, ret: Record<string, any>) => {
      ret.id = ret._id.toString();
      delete ret._id;
      delete ret.__v;
    },
  },
})
export class Drink {
  @Prop({ required: true })
  name!: string;

  @Prop({ type: [ToppingSchema], default: [] })
  toppings!: Topping[];
}
export type DrinkDocument = Drink & Document;
export const DrinkSchema = SchemaFactory.createForClass(Drink);

```

Module wiring

```

// drink.module.ts
@Module({
  imports: [
    MongooseModule.forFeature([{ name: Drink.name, schema: DrinkSchema }]),
  ],
  controllers: [DrinkController],
  providers: [DrinkService],
})
export class DrinkModule {}

```

```

// app.module.ts
@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    MongooseModule.forRootAsync({
      inject: [ConfigService],
      useFactory: (config: ConfigService) => ({
        uri: config.get<string>('MONGODB_URI'),
      }),
    }),
    DrinkModule,
  ],
})
export class AppModule {}

```

Service + Controller — bộ khung tối giản

```

@Injectable()
export class DrinkService implements OnModuleInit {
  constructor(
    @InjectModel(Drink.name) private readonly drinkModel:
    Model<DrinkDocument>,
  ) {}

  async onModuleInit(): Promise<void> {
    const count = await this.drinkModel.countDocuments();
    if (count === 0) {
      await this.drinkModel.insertMany(SEED_DRINKS);
    }
  }

  findAll(keyword?: string): Promise<Drink[]> {
    const key = keyword?.trim();
    if (!key) return this.drinkModel.find().exec();
    return this.drinkModel
      .find({ name: { $regex: key, $options: 'i' } })
      .exec();
  }

  async findOne(id: string): Promise<Drink> {
    const drink = Types.ObjectId.isValid(id)
      ? await this.drinkModel.findById(id).exec()
      : null;
    if (!drink) throw new NotFoundException(`Drink with id ${id} not
found`);
    return drink;
  }
}

```

```

@Controller('drinks')
export class DrinkController {
  constructor(private readonly drinkService: DrinkService) {}

  @Post()
  create(@Body() dto: CreateDrinkDto) {
    return this.drinkService.create(dto);
  }

  @Get()
  findAll(@Query('keyword') keyword?: string) {
    return this.drinkService.findAll(keyword);
  }

  @Get('/:id')
  findOne(@Param('id') id: string) {
    return this.drinkService.findOne(id);
  }
}

```

```

@Patch('/:id')
update(@Param('id') id: string, @Body() dto: UpdateDrinkDto) {
  return this.drinkService.update(id, dto);
}

@Delete('/:id')
@HttpCode(204)
async remove(@Param('id') id: string) {
  await this.drinkService.remove(id);
}
}

```

Repository Pattern — bộ khung tối giản

```

// drink.repository.interface.ts
export interface DrinkRepository {
  findAll(keyword?: string): Promise<Drink[]>;
  findOne(id: string): Promise<Drink | null>;
}
export const DRINK_REPOSITORY = Symbol('DRINK_REPOSITORY');

// mongoose-drink.repository.ts
@Injectable()
export class MongooseDrinkRepository implements DrinkRepository {
  constructor(@InjectModel(Drink.name) private readonly drinkModel:
Model<DrinkDocument>) {}
  findAll(keyword?: string) {
    /* ...giống hệt code cũ trong service... */
  }
  findOne(id: string) {
    return Types.ObjectId.isValid(id) ? this.drinkModel.findById(id).exec()
: Promise.resolve(null);
  }
}

// drink.module.ts
providers: [DrinkService, { provide: DRINK_REPOSITORY, useClass:
MongooseDrinkRepository }],

// drink.service.ts
constructor(@Inject(DRINK_REPOSITORY) private readonly repo:
DrinkRepository) {}

```

Những cái bẫy hay dính

Bẫy	Sai	Đúng
Quên <code>@InjectModel</code>	<code>constructor(private readonly m: Model<T>)</code>	<code>constructor(@InjectModel(X.name) private readonly m: Model<T>)</code>
Id parse bằng <code>+id</code>	<code>findOne(+id)</code>	<code>findOne(id)</code> — id là chuỗi ObjectId, không phải số
Không kiểm tra id trước khi query	<code>findById(id)</code> thẳng	<code>Types.ObjectId.isValid(id) ? findById(id) : null</code>
Seed vô điều kiện	<code>insertMany(SEED)</code> mỗi lần khởi động	<code>if (count === 0) insertMany(SEED)</code>
Subdocument thiếu <code>_id: false</code>	<code>@Schema()</code> cho subdocument	<code>@Schema({ _id: false })</code>
Quên <code>toJSON.transform</code>	client nhận <code>_id, __v</code>	transform <code>_id</code> → <code>id</code> , xóa <code>__v</code>
Đọc env trực tiếp	<code>process.env.MONGODB_URI</code> trong <code>useFactory</code>	<code>inject: [ConfigService], config.get(...)</code>
Quên <code>await</code> ở <code>remove</code>	<code>this.service.remove(id)</code> không <code>await</code>	<code>await this.service.remove(id)</code>
Bind sai token	<code>{ provide: DrinkService, useClass: Repo }</code>	<code>{ provide: DRINK_REPOSITORY, useClass: Repo }</code>
Khai kiểu bằng class thay vì interface	<code>private readonly repo: MongooseDrinkRepository</code>	<code>private readonly repo: DrinkRepository</code>

Lệnh test nhanh

```
# Toàn bộ CRUD trong một hơi
curl -s http://localhost:3001/drinks | python3 -m json.tool
curl -s -X POST http://localhost:3001/drinks -H "Content-Type: application/json" -d '{...}'
curl -s -X PATCH http://localhost:3001/drinks/<id> -H "Content-Type: application/json" -d '{...}'
curl -s -X DELETE http://localhost:3001/drinks/<id> -w "\nstatus: %{http_code}\n"
curl -s -w "\nstatus: %{http_code}\n" http://localhost:3001/drinks/id-sai-dinh-dang
```